

EVO AURA BILLING SOFTWARE

Codex Refactor, File Alteration, Windows 10/11 EXE and Installer Guide

IMPORTANT TARGET RULE: Build and optimize only for Windows 10 and Windows 11. Do not downgrade or change the project for Windows 7 or Windows 8 compatibility.

Prepared for editing the current available files and sending to Codex.
All files shown in your folder screenshot are included in this plan.

1. Goal of this PDF

This PDF is a direct instruction document for Codex. It explains how to alter the current Evo Aura billing software files, keep the existing files, create missing pages, make per-computer database creation work correctly inside an installed EXE, and package the app for Windows 10/11 without unnecessary legacy Windows 7/8 changes.

- Do not remove any existing file from the project response or plan.
- Keep PyQt6 as the main GUI stack for Windows 10/11 and Windows 11.
- Make the installed EXE create writable company databases outside the installation folder.
- Use a one-folder PyInstaller build for speed, then wrap that folder with an installer.
- Make startup, navigation, product loading, report loading, and table filtering smooth without lag.

2. Existing files and how Codex must handle them

These are the files and folders visible in your screenshot. Codex must not ignore them. Some are source files, some are generated files, and some are assets or development folders.

File / Folder	Purpose	Required action
evoaura_app.py	Main application / startup auth / DB / app shell	Keep. Refactor carefully. Add app-data database paths, page registry, and packaging-safe resource paths.
loyalty_page.py	Customer loyalty page	Keep. Wire to sidebar route key loyalty and ensure it accepts db_name/current_user.
saravana.db	Existing SQLite database file	Keep as development/sample data. Do not write production DBs into install folder. New installed computers create DBs in AppData.
__pycache__	Python generated cache folder	Do not edit manually. Add to .gitignore and do not include intentionally in installer.
credit_management_page.py	Due collection / credit page	Keep. Route key credit should open this page.
sidebar.py	Apple-style collapsible sidebar	Keep. Ensure route keys match app registry. Update icon loading for PyInstaller resource paths.
customer_page.py	Customer center page	Keep. Route key customer or customers should open this page consistently.
low_stock_page.py	Reorder control / low stock page	Keep. Optimize DB queries and route key low_stock.
purchase_orders_page.py	Purchase invoices / purchase order page	Keep. Route key purchase_orders.
dashboard.py	Dashboard page file	Keep. Use as DashboardPage instead of duplicate dashboard code inside main file where possible.
supplier_page.py	Supplier page	Keep. Route key suppliers. Optimize table and filters.
product_page.py	Main product page	Keep. This is large and important. Optimize loading, table model, DB indexes, and image thumbnail cache.
.git	Git repository folder	Keep for source control only. Never include .git in installer.
app_branding_qt5.py	Older Qt5 branding helper	Keep as legacy backup only. Do not import in PyQt6 app unless you intentionally support Qt5.
app_branding.py	Current branding/icon helper	Keep. Make PyInstaller-safe path handling.
input_behavior_qt5.py	Older Qt5 input behavior helper	Keep as legacy backup only. Do not import in PyQt6 app.
input_behavior.py	Current input behavior helper	Keep. Make sure it is safe for PyQt6 and does not slow typing.
invoice.pdf	Sample/generated invoice PDF	Keep for reference. Generated invoices should save to a user folder, not to Program Files.
icons	Icon assets folder	Keep and include in PyInstaller add-data. Add app .ico here if missing.

3. Current app flow to preserve

The current main application already has a good setup/authentication flow. Preserve the behavior, but change where writable files are stored after installation.

Start app

- > Find existing Evo Aura DB in user data folder
- > If no DB is selected or no company DB exists:
AskDBName -> MasterAuth -> CompanySetup -> Signup ADMIN -> QR Setup -> Login
- > If DB exists but no users:
Signup ADMIN -> QR Setup -> Login
- > If DB exists and users exist:
Login -> OTP / Recovery code -> Dashboard with Sidebar

- On every new computer, the installed EXE must be able to create a new local company database.
- The installer folder may be read-only. Never create or update databases inside Program Files or the PyInstaller dist folder.
- Existing development DB saravana.db can remain in the project, but production DBs must live in the user data folder.

4. Windows target rules

Item	Decision
Operating system	Windows 10 and Windows 11 only. Do not change code for Windows 7 or Windows 8.
GUI stack	Use PyQt6. Do not downgrade to PyQt5 for old Windows support.
Python	Use Python 3.11 or 3.12 64-bit for the build environment.
Installer style	Use PyInstaller one-folder build, then Inno Setup / NSIS installer.
Database location	%LOCALAPPDATA%\EvoAura\Data\<company>.db
Invoice output location	Documents\EvoAura\Invoices or %LOCALAPPDATA%\EvoAura\Invoices
Logs / config location	%LOCALAPPDATA%\EvoAura\logs and %LOCALAPPDATA%\EvoAura\config.json

5. File-by-file alteration plan for Codex

5.1 evoaura_app.py

- Keep this as the main launcher for the first refactor pass. Later it may be split into main.py, db.py, auth pages, and app_shell.py.
- Replace `os.listdir(".")` DB discovery with a user-data DB discovery function that checks `%LOCALAPPDATA%\EvoAura\Data`.
- Keep the current auth flow, but make DB paths absolute and writable.
- Add `resource_path()` for bundled assets and `app_data_dir()` for writable files.
- Move hardcoded paths, invoice paths, and DB paths away from the project root.
- Add imports or lazy imports for `dashboard.py` and `loyalty_page.py` because both files exist in the current project.
- Make a single `PAGE_REGISTRY` mapping nav keys to page builder functions.
- Do not create all pages at startup. Keep lazy loading so app opens fast.
- Fix route aliases: `product/products`, `customer/customers`, `pl_summary/pl_summaries`, `stock_report/stock_reports`, `security/securitys`.

- Move MASTER_SECRET into config or environment for release. If kept for development, add a TODO and do not print it in UI.

5.2 sidebar.py

- Keep the current collapsible sidebar design and animation.
- Do not downgrade for Windows 7/8. Keep PyQt6 imports.
- Ensure NAV_ITEMS keys exactly match the app PAGE_REGISTRY.
- Fix typo keys when possible: pl_summarys -> pl_summary, securitys -> security. If existing code depends on old keys, support aliases too.
- Use resource_path("icons") instead of os.path.dirname(__file__) only, so icons load inside PyInstaller.
- Cache tinted QPixmaps or icons to reduce lag when hovering/collapsing sidebar.
- Keep version text Evo Aura v1.0, but make it a constant APP_VERSION.

5.3 dashboard.py

- Keep this file and convert/confirm its main class is DashboardPage(QWidget).
- Use this file for the home page instead of maintaining duplicate _DashboardPage code inside evoaura_app.py.
- Dashboard should accept db_name, current_user, and navigate_cb.
- Load dashboard KPI counts using fast SQL queries, not full table scans.
- Do not block startup with heavy charts. Load charts after page appears using QTimer or a worker thread.

5.4 product_page.py

- Keep this as the main product master page.
- Because it is a large file, do not load every product image and every row at once.
- Use QTableView + QAbstractTableModel or paginated loading for large inventory.
- Add indexes for item_code, sku, barcode, product_name, category, supplier_id, stock_status.
- Make db_name an absolute database path passed from evoaura_app.py.
- Use thumbnails for product images and cache them.
- Search box should use delayed search, for example 250 ms QTimer debounce, to prevent typing lag.
- Add textile-ready variant logic: product master, product_variant table, color, size, barcode, stock_qty.

5.5 supplier_page.py

- Keep this as supplier management.
- Accept db_name/current_user from the shell.
- Use reusable KPI cards and advanced filter/table view.
- Avoid reloading the full table after every small edit; refresh only affected rows where possible.
- Connect supplier IDs to product and purchase invoice tables.

5.6 purchase_orders_page.py

- Keep this file for Purchase Invoices.
- Use transaction-safe save: purchase_invoice header, purchase_invoice_items, stock movement, supplier ledger update.
- After saving purchase, update product_variant stock, not just product master stock.
- PDF or print generation should run after DB commit and should not freeze the UI.
- Route key must be purchase_orders.

5.7 low_stock_page.py

- Keep this file for Reorder Control.
- Read from product_variant and product stock tables with indexed queries.
- Show current stock, available stock, reorder level, supplier, last purchase date, and actions.
- Add quick action to create purchase invoice from low-stock selected items.
- Do not recalculate everything on every repaint.

5.8 customer_page.py

- Keep this file for Customer Center.
- Route aliases customer and customers should both open it.
- Connect customer to sales, due collection, loyalty, and ledger.
- Use search debounce and paginated table loading.
- Add basic fields: name, mobile, address, GSTIN if business customer, opening balance, credit limit.

5.9 credit_management_page.py

- Keep this file for Due Collection.
- Route key credit should open it.
- Track dues from sales invoices and payments.
- Use customer_ledger or payments table; do not store only a single due value without history.
- Add receipt printing or PDF receipt later.

5.10 loyalty_page.py

- Keep this file because it exists in the current folder.
- Import it lazily in evoaura_app.py and map route key loyalty to LoyaltyPage.
- Connect loyalty points to customer_id and sales invoice total.
- Start simple: points earned, points redeemed, current balance, manual adjustment, history table.

5.11 app_branding.py and app_branding_qt5.py

- Use app_branding.py for the PyQt6 app.
- Keep app_branding_qt5.py only as a legacy backup. Do not import it from PyQt6 modules.
- Make apply_app_icon safe when running from source and when frozen by PyInstaller.
- If icons/app.ico does not exist, create it from the main logo and place it in icons.

5.12 input_behavior.py and input_behavior_qt5.py

- Use input_behavior.py for the PyQt6 app.
- Keep input_behavior_qt5.py only as a legacy backup.
- Make the global input guard lightweight. It must not slow typing in product search or billing search fields.
- Avoid installing repeated event filters every time a page is opened.

5.13 icons folder

- Keep all existing PNG icons.
- Add an app icon file icons/evoaura.ico for EXE and installer.
- Include icons in PyInstaller using --add-data "icons;icons" on Windows.
- In code, load icons through resource_path("icons/<name>.png").

5.14 saravana.db

- Keep this database file for development/testing only.
- Do not use it as the default production DB on installed computers.
- If it contains real customer/business data, do not ship it inside the public installer.
- If you want a demo mode, copy it as sample_saravana.db and open it only when user chooses demo.

5.15 invoice.pdf

- Keep as a sample invoice/reference file.
- Generated invoices from the EXE should go to Documents/EvoAura/Invoices or AppData invoices folder.
- Never overwrite the reference invoice.pdf in the install directory.
- Invoice PDF generation should run in a worker or after Save so billing UI does not freeze.

5.16 .git and __pycache__

- .git stays in source project only and is not included in installer.
- __pycache__ is generated by Python. Do not edit it manually.
- Add __pycache__/, *.pyc, build/, dist/, *.spec if desired, and local DB files to .gitignore depending on your backup policy.
- Keep the files in this response as requested, but do not pack .git and __pycache__ into the installer.

6. Sidebar route registry to implement

Every sidebar key must open either a real page or a clean placeholder page. Missing pages should be created one by one, but the app must never crash when a menu item is clicked.

Route key	File	Class	Status/action
home	dashboard.py	DashboardPage	Existing file - wire it
new_sale	new_sale_page.py	NewSalePage	Create
returns	returns_page.py	ReturnsPage	Create
bill_view	bill_view_page.py	BillViewPage	Create
product / products	product_page.py	ProductPage	Existing
purchase_orders	purchase_orders_page.py	PurchaseOrdersPage	Existing
suppliers	supplier_page.py	SupplierPage	Existing
low_stock	low_stock_page.py	LowStockPage	Existing
customer / customers	customer_page.py	CustomerPage	Existing
credit	credit_management_page.py	CreditManagementPage	Existing
loyalty	loyalty_page.py	LoyaltyPage	Existing
pl_summary / pl_summarys	pl_summary_page.py	PLSummaryPage	Create
expense	expense_page.py	ExpensePage	Create
gst	gst_breakdown_page.py	GSTBreakdownPage	Create
cashflow	cashflow_page.py	CashFlowPage	Create
daybook	daybook_page.py	DayBookPage	Create
stock_report / stock_reports	stock_report_page.py	StockReportPage	Create
trial_balance	trial_balance_page.py	TrialBalancePage	Create
export	export_page.py	ExportPage	Create
company	company_settings_page.py	CompanySettingsPage	Can reuse current settings UI
users_roles	users_roles_page.py	UsersRolesPage	Create
security / securitys	security_page.py	SecurityPage	Create
audit	audit_log_page.py	AuditLogPage	Create

7. Required source changes for installed EXE data paths

Codex must add a small path utility and then use it everywhere. Bundled resources are read-only. Databases and generated invoices are writable user data.

```
# paths.py or inside evoaura_app.py for first pass
from pathlib import Path
import os, sys

APP_NAME = "EvoAura"

def resource_path(relative_path: str) -> Path:
    """Read-only bundled resource path for icons, logo, templates."""
    if getattr(sys, "frozen", False):
        base = Path(getattr(sys, "_MEIPASS", Path(sys.executable).parent))
    else:
        base = Path(__file__).resolve().parent
    return base / relative_path
```

```
def app_data_dir() -> Path:
    """Writable per-user data folder. Works in installed EXE."""
    root = os.getenv("LOCALAPPDATA") or str(Path.home())
    path = Path(root) / APP_NAME
    path.mkdir(parents=True, exist_ok=True)
    return path

def db_dir() -> Path:
    path = app_data_dir() / "Data"
    path.mkdir(parents=True, exist_ok=True)
    return path

def invoice_dir() -> Path:
    docs = Path.home() / "Documents" / APP_NAME / "Invoices"
    docs.mkdir(parents=True, exist_ok=True)
    return docs

def safe_db_path(db_name: str) -> Path:
    clean = db_name.strip().replace(" ", "_")
    if not clean.endswith(".db"):
        clean += ".db"
    return db_dir() / clean
```

- In `_boot()`, scan `db_dir()` instead of the current working directory.
- In `_on_db_name()`, call `safe_db_path(db_name)` before checking exists/opening.
- Pass the absolute DB path to every page as `db_name` or `db_path`.
- Use `resource_path("icons/home.png")` for icons and `resource_path("icons/evoaura.ico")` for app icon.
- Generated invoices must go through `invoice_dir()`.

8. Database tables to migrate toward

Do not try to finish every business feature in one edit. But the database should be designed so sales, textile variants, purchase, stock, customer due, and reports work correctly.

Table	Purpose
company	Company profile, logo, GST, footer
users	Login users, role, TOTP, recovery codes
roles / permissions	Optional advanced access control
suppliers	Supplier master
customers	Customer master and credit settings
products	Product master: name, category, brand, tax, base info
product_variants	Barcode/SKU/color/size/fabric/fit/MRP/selling price/current stock
stock_movements	Every stock in/out/adjustment with reference
purchase_invoices	Purchase header
purchase_invoice_items	Purchase item lines
sales_invoices	Sales header
sales_invoice_items	Sales item lines
payments	Cash/card/UPI/credit/partial payments
customer_ledger	Due collection and customer balance history
expenses	Expense tracking
loyalty_transactions	Earn/redeem/manual loyalty changes
audit_log	User activity, edits, deletes, login, security events

9. Smooth performance and no-lag workflow

Area	Instruction
Startup	Load auth and dashboard only. Do not import all heavy pages at startup. Use lazy imports.
Navigation	Create page on first open. Reuse page if light; discard/recreate only heavy pages intentionally.
Tables	Prefer QTableView + model or pagination. Avoid loading 10,000+ rows into QTableWidgetItem at once.
Search	Use QTimer debounce around 200-300 ms before SQL query.
Images	Load thumbnails only. Cache pixmaps. Do not store full image widgets in every table cell.
Reports	Run heavy report queries/PDF generation in QThread or worker, not UI thread.
SQLite	Use indexes, WAL mode, parameterized queries, and transactions.
PyInstaller	Use one-folder build for faster startup. Avoid onefile for daily POS use.
QSS	Apply app stylesheet once. Do not rebuild giant stylesheets repeatedly while typing.
Icons	Cache tinted icons in sidebar to reduce hover animation work.

10. Convert to EXE - Windows 10/11 build procedure

Use one-folder PyInstaller build first. It starts faster and is better for a billing/POS desktop app. After that, create an installer from the dist folder.

10.1 Create clean build environment

```
# Run in project folder on Windows 10/11
py -3.11 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip setuptools wheel
pip install PyQt6 pyotp qrcode[pil] Pillow pyinstaller
```

10.2 Add/confirm an app icon

- Put the installer/EXE icon at icons/evoaura.ico.
- If only PNG exists, convert the logo PNG to .ico before building.
- Keep all existing sidebar PNG icons in the icons folder.

10.3 PyInstaller one-folder command

```
pyinstaller ^
--noconfirm ^
--clean ^
--windowed ^
--name "EvoAura" ^
--icon "icons\evoaura.ico" ^
--add-data "icons;icons" ^
--collect-submodules PyQt6 ^
--collect-data PyQt6 ^
evoaura_app.py
```

- If the entry file is renamed later to main.py, change the last line to main.py.
- Test dist/EvoAura/EvoAura.exe before creating installer.

- Do not use `--onefile` for the final POS app unless you accept slower startup due to extraction.

10.4 Optional .spec file notes

```
# In EvoAura.spec, make sure datas includes icons:
datas=[('icons', 'icons')]

# Keep console disabled for GUI app:
console=False

# Keep one-folder COLLECT output for faster launch:
# Build with: pyinstaller EvoAura.spec --clean --noconfirm
```

10.5 Installer using Inno Setup

```
#define MyAppName "Evo Aura"
#define MyAppVersion "1.0.0"
#define MyAppExeName "EvoAura.exe"

[Setup]
AppId={{EVO-AURA-BILLING-SOFTWARE}}
AppName={#MyAppName}
AppVersion={#MyAppVersion}
DefaultDirName={autopf}\Evo Aura
DefaultGroupName=Evo Aura
OutputDir=installer
OutputBaseFilename=EvoAura_Setup
Compression=lzma
SolidCompression=yes
WizardStyle=modern
ArchitecturesInstallIn64BitMode=x64

[Files]
Source: "dist\EvoAura\*"; DestDir: "{app}"; Flags: ignoreversion recursesubdirs createallsubdirs

[Icons]
Name: "{group}\Evo Aura"; Filename: "{app}\{#MyAppExeName}"
Name: "{commondesktop}\Evo Aura"; Filename: "{app}\{#MyAppExeName}"; Tasks: desktopicon

[Tasks]
Name: "desktopicon"; Description: "Create a desktop shortcut"; GroupDescription: "Additional icons:"

[Run]
Filename: "{app}\{#MyAppExeName}"; Description: "Launch Evo Aura"; Flags: nowait postinstall skipifsilent
```

- The installer installs program files to Program Files, but the EXE writes DB/config/invoices to AppData/Documents.
- Do not install saravana.db as the active production DB unless you intentionally create a demo installer.
- Do not include .git or __pycache__ in installer.

11. First-run workflow after installation

Fresh Windows 10/11 computer:

1. User installs Evo Aura.
 2. User opens EvoAura.exe.
 3. App creates %LOCALAPPDATA%\EvoAura\Data if missing.
 4. App checks for existing selected DB / company DB.
 5. If none, AskDBName page appears.
 6. User enters company/database name, for example Saravana.
 7. App creates %LOCALAPPDATA%\EvoAura\Data\Saravana.db.
 8. MasterAuth validates setup authorization.
 9. CompanySetup saves company info.
 10. Signup creates ADMIN user and TOTP secret.
 11. QR setup shows authenticator QR.
 12. Login + OTP opens dashboard.
- This makes the same EXE installer work on every new computer.
 - Every computer can create its own local DB without needing the source code folder.
 - Backup/restore can be added later by copying DB files from AppData/Data.

12. Recommended organization without losing existing files

First, make the current flat-folder project run correctly. After that, Codex may organize the project. If files are moved into folders, update imports carefully and test after each move.

```
EvoAura/  
  evoaura_app.py          # current entry; later can become main.py  
  sidebar.py  
  dashboard.py  
  product_page.py  
  supplier_page.py  
  purchase_orders_page.py  
  low_stock_page.py  
  customer_page.py  
  credit_management_page.py  
  loyalty_page.py  
  app_branding.py  
  input_behavior.py  
  app_branding_qt5.py      # legacy backup only  
  input_behavior_qt5.py    # legacy backup only  
  saravana.db              # dev/sample only  
  invoice.pdf              # sample/reference only  
  icons/  
  .git/  
  __pycache__/  
  
# missing pages to create:  
new_sale_page.py  
returns_page.py  
bill_view_page.py  
pl_summary_page.py  
expense_page.py
```

gst_breakdown_page.py
cashflow_page.py
daybook_page.py
stock_report_page.py
trial_balance_page.py
export_page.py
company_settings_page.py
users_roles_page.py
security_page.py
audit_log_page.py

13. Standard page constructor rule

To keep the shell consistent, every page should accept the same common arguments even if not all are used.

```
class SomePage(QWidget):
    def __init__(self, db_name: str = "", current_user: str = "Admin", navigate_cb=None, parent=None):
        super().__init__(parent)
        self.db_name = db_name
        self.current_user = current_user
        self.navigate_cb = navigate_cb
        self._build_ui()
```

- Use db_name as the absolute database path.
- Do not open a different DB inside each page using relative paths.
- When possible, centralize DB connection helpers to avoid duplicated SQLite code.

14. Acceptance checklist for Codex

- [] App launches from source on Windows 10/11.
- [] App launches from dist/EvoAura/EvoAura.exe on Windows 10/11.
- [] Fresh install with no DB opens AskDBName and creates DB in %LOCALAPPDATA%/EvoAura/Data.
- [] No database or invoice is written into Program Files or the dist folder.
- [] Sidebar opens every route without crashing, even for placeholder pages.
- [] Existing files product_page.py, supplier_page.py, purchase_orders_page.py, low_stock_page.py, customer_page.py, credit_management_page.py, loyalty_page.py, dashboard.py are all wired.
- [] Icons load both from source and from installed EXE.
- [] Product search does not lag while typing.
- [] Large product table does not freeze UI on page open.
- [] PDF invoice generation does not freeze billing screen.
- [] PyInstaller build uses one-folder and then installer wraps the folder.
- [] No Windows 7/8 compatibility downgrade was added.

15. Copy-paste instruction for Codex

Use this as the main prompt when you send the project and this PDF to Codex.

You are editing my Evo Aura Billing Software PyQt6 project for Windows 10/11 only.

Do not downgrade or modify for Windows 7/8.

Do not ignore existing files. Keep and wire these files:

Evo Aura Billing Software - Codex Alteration and EXE Guide

evoaura_app.py, sidebar.py, dashboard.py, product_page.py, supplier_page.py, purchase_orders_page.py, low_stock_page.py, customer_page.py, credit_management_page.py, loyalty_page.py, app_branding.py, input_behavior.py, app_branding_qt5.py, input_behavior_qt5.py, icons folder, saravana.db reference DB, invoice.pdf reference file, .git folder, and __pycache__ generated folder.

Main tasks:

1. Preserve current startup/auth flow.
2. Make installed EXE create and use databases under %LOCALAPPDATA%\EvoAura\Data.
3. Never write DB/invoice/config files into the install folder.
4. Add resource_path() and app_data_dir() handling for PyInstaller.
5. Wire all existing pages to sidebar routes, including dashboard.py and loyalty_page.py.
6. Create missing placeholder pages for all sidebar routes not yet implemented.
7. Add a clean PAGE_REGISTRY with route aliases.
8. Optimize performance: lazy page loading, table pagination/model, search debounce, icon cache, image thumbnail cache, worker threads for reports/PDF generation.
9. Prepare PyInstaller one-folder build and Inno Setup installer instructions.
10. Test from source and from dist/EvoAura/EvoAura.exe on Windows 10/11.

16. Final implementation order

Order	Task
1	Fix app-data DB path and resource path first.
2	Make source run without missing imports.
3	Wire dashboard.py, loyalty_page.py, and all current pages.
4	Add missing placeholder pages for unbuilt menu routes.
5	Optimize product/supplier/customer tables.
6	Add DB schema migrations and stock movement tables.
7	Build PyInstaller one-folder EXE.
8	Create installer and test on a clean Windows 10/11 system.